



Coordinated Adaptive Traffic Signal Control

Case Study Report

Supervisor: Dr. Nghiem Thi Phuong

Group members:

Nguyen Nhu Tien Tuan	22BA13314
Tran Hien Chuong	22BA13056
Do Gia Bao	22BA13042
Bui The Anh	22BA13007
Le Duc An	22BA13002
Pham Quang Tung	22BA13320

January 19, 2026

Abstract

Traffic jams in big cities like Hanoi and Ho Chi Minh City have become a serious problem. They cause big money losses and make air pollution worse. Old traffic light systems, which use fixed times, are easy to set up but no longer work well. Traffic today changes quickly and is often hard to predict.

To solve this, a new smart traffic light system was created using Reinforcement Learning (RL), a type of Artificial Intelligence. The system is trained in a traffic simulator called SUMO, where it learns how to control traffic lights by testing many different traffic situations.

What makes this project special is that the AI model is not just used in simulation. It is also built into a real software application. This app lets users set up, watch, and check how the smart traffic lights are working. The simulation results show that the AI-controlled lights help cars wait less compared to old fixed-time systems. This tool brings AI closer to real-world use and shows how smart technology can make traffic in cities faster and cleaner.

Contents

1	Introduction	3
1.1	Background	3
1.2	Problem statement	3
1.3	Objectives and scope	4
1.4	Report structure	4
2	Background and related work	6
2.1	Traffic signal control approaches	6
2.2	Reinforcement learning for traffic signal control	6
2.3	Simulation using SUMO and TraCI	7
2.4	Reward design patterns in the literature	8
3	Methodology and system design	9
3.1	Simulation Environment and System Setup	9
3.1.1	Software Dependencies	9
3.1.2	Integration with SUMO	9
3.1.3	Scenario Configuration	10
3.1.4	System Verification	11
3.2	Training Methodology	11
3.2.1	MDP specification	11
3.2.2	Action space and constraints	13
3.2.3	Reward function	14
3.2.4	Learning algorithm and network architecture	16
3.2.5	Curriculum Strategy	17
3.2.6	Training Procedure Workflow	18
3.3	Evaluation Methodology	18
3.3.1	Baselines	19
3.3.2	Evaluation Procedure Workflow	19
3.3.3	Execution Pipeline for Evaluation	19
4	Results and Discussion	20

Chapter 1

Introduction

1.1 Background

Traffic lights play a very important role in city traffic because they affect how long cars wait, how fast queues grow, how many cars get through an intersection, and even things like fuel use and pollution. In fast-growing cities, traffic is not the same all the time. It can change during the day, between weekdays and weekends, and also because of accidents, bad weather, road work, or other sudden events.

In these situations, old-style fixed-time traffic lights—even when set up carefully using past data—can stop working well very quickly. Some systems use sensors or timing plans to adjust the lights a bit better, but these still need people to guess traffic patterns and often require a lot of manual changes and updates.

Reinforcement Learning (RL) offers a smart solution by letting the system learn how to control traffic lights over time. RL works by treating the problem like a game: the system makes a decision, sees what happens, and gets a reward based on the result [4]. Over time, it learns what works best.

Many studies now show that RL-based traffic control can cut down wait times and reduce long queues better than traditional systems, especially when the AI is given useful information and a reward that matches real-world goals [31]. These strong results support the idea of building a full system—not just a test model—but a working tool that people can set up, watch in action, and measure for real use.

1.2 Problem statement

The main problem this case study focuses on is building a smart traffic light controller that works well even when traffic keeps changing. The goal is to improve how intersections perform when traffic is high, low, or unpredictable. At the same time, the system must be practical. It needs to be easy to train in simulation, simple to test, and ready to

use in a real software application. The challenge is not just to create an AI model that works well in simulation. It's also important to clearly define how the traffic control problem is structured, using something called a Markov Decision Process (MDP). This includes setting up how the system sees traffic, what actions it can take, and how it gets feedback for making good decisions. The action choices must follow real traffic rules, and the reward should help reduce waiting time without causing problems like traffic spilling into other roads or the lights switching too often. Also, the system should include tools that let users monitor and evaluate its performance. This allows for a fair comparison with traditional traffic signal systems under the same testing conditions. Because of all this, the project focuses on a full end-to-end solution. It trains the AI inside a detailed simulator and then puts the trained model into a real application for testing and use. This full process matches what many researchers in reinforcement learning for traffic control (RL-TSC) have seen: the final results depend a lot on how the system is built, including how the data is prepared, how rewards are given, how stable the training is, and how the traffic scenarios are designed [31].

1.3 Objectives and scope

The objectives of the study are to develop a SUMO-based environment controlled through TraCI [3], formulate the decision process as an MDP with a clear state, action, reward, and discounting scheme [4], train a deep RL controller that can select signal actions robustly across varying traffic conditions, and provide a usable application layer that supports configuration, monitoring, and evaluation. The scope is simulation-based evaluation, not field deployment. The system is intended as a structured prototype that can be extended toward real-world deployment after additional validation, sensing integration, and safety engineering.

1.4 Report structure

Chapter 2 gives background information to help understand the project. It explains how traffic signal control works, how reinforcement learning (RL) can be used for traffic light systems, and how simulation tools like SUMO and TraCI are used in this kind of research. Chapter 3 describes the method and system design in detail. It includes a full explanation of the Markov Decision Process (MDP), how the traffic data is prepared and normalized, the rules and limits for choosing actions, and how the reward is calculated. It also talks about time-based discounting, the learning algorithm used, the baseline methods for comparison, how the traffic scenarios are built, and how the training and testing are done. Finally, it explains how the trained model is added to the application

for real use. Chapter 4 presents the results from the experiments and discusses what they mean in real-world situations. The bibliography at the end lists all the sources and research papers that were used in this study.

Chapter 2

Background and related work

2.1 Traffic signal control approaches

Fixed-time traffic control is still commonly used today because it is easy to understand, works in a predictable way, and doesn't need much technology to run. These systems are usually set up using offline calculations or expert rules, which are based on the idea that traffic demand stays mostly the same throughout the day. In some areas, traffic lights are timed to work together across several intersections, which helps cars move more smoothly along main roads. However, this type of coordination does not work well when traffic patterns suddenly change.

Actuated traffic control is a more flexible method. It uses sensors to detect vehicles and adjusts the lights based on what it sees. Still, it usually depends on fixed rules created by engineers, and it only uses local information from the sensors at each intersection.

Some older adaptive systems, like SCOOT and OPAC, try to be smarter by using models or optimization methods to adjust the traffic signals automatically. These systems can work well but need a lot of setup and tuning. They can also become very complex when used in larger networks or when traffic behavior doesn't match the original assumptions used in the model [18, 19, 20, 21].

2.2 Reinforcement learning for traffic signal control

Reinforcement Learning (RL) sees traffic signal control as a step-by-step decision-making process. In this setup, an AI agent watches the current traffic situation and decides what action to take, such as choosing the next traffic light phase, how long each phase should last, or how to set the full light cycle. Some early research showed that RL could work well for smart traffic lights, including systems with multiple intersections working together [8, 10, 23].

Queue length and delay are often used as learning signals in these systems because

they clearly show how bad congestion is and how long drivers have to wait. For example, Genders and Razavi used both delay and queue information to help the AI learn [7], while van der Pol and Oliehoek used similar ideas for systems where different traffic lights work together [25].

As deep RL methods became easier to use, more advanced approaches like DQN (Deep Q-Network) [13] and multi-agent deep RL [5, 12, 33] were applied to larger traffic networks with more complex control needs.

Many review papers point out that how well RL works depends on several important factors. These include how the traffic state is represented, how the reward is designed, how stable the training process is, and how realistic the traffic simulator is [31]. Other studies also warn that without proper limits or penalties, the AI can learn unstable behaviors, such as switching the lights too often, which makes traffic worse [11]. Because of these challenges, it is important to design the system carefully. That means clearly describing the MDP, using data normalization to help the learning process stay stable, and making sure the reward system balances both efficiency and smooth operation.

2.3 Simulation using SUMO and TraCI

SUMO is a popular traffic simulator used in research on traffic signal control [3, 28]. It is a microscopic simulator, meaning it looks at each vehicle and models how they move in detail. SUMO is often used with a tool called TraCI, which allows researchers to connect their own programs to the simulator. Through TraCI, the AI agent can read traffic data—like how many cars are waiting, how long they have waited, and how full the lanes are—and then decide what actions to take. These actions include choosing traffic light cycles and deciding how long each green light should last. Of course, the system still needs to follow traffic safety rules, like giving enough time for yellow and all-red signals between phases.

One important thing to keep in mind when using SUMO is something called teleportation. This is a special feature in the simulator that helps prevent traffic deadlocks. When traffic becomes extremely jammed and vehicles can't move anymore, SUMO may "teleport" some vehicles—meaning it removes or moves them somewhere else—so that the simulation can keep running. In real life, this kind of teleportation doesn't happen, so it's not part of the goal. However, it is a useful warning sign that something is going wrong in the simulation, like very bad traffic or incorrect setup.

SUMO provides information about when and why teleportation happens, and how users can change the settings if needed [37]. In experiments using reinforcement learning, researchers often keep track of how many vehicles are teleported. Too much teleportation can show that the traffic system isn't working well, so it might be counted as a penalty

or managed through the simulation settings to make the system more realistic and avoid total gridlock.

2.4 Reward design patterns in the literature

In reinforcement learning for traffic control, reward design links traffic goals to learning. A common method is using a negative reward based on delay or waiting time—IntelliLight, for example, uses total vehicle delay as a learning signal [30]. Queue length is also widely used, as it clearly reflects congestion and is easy to observe [25]. Some studies combine both delay and queue length for better performance [7].

To reduce spillback and blockage, pressure-based rewards encourage actions that prevent downstream overload. Max-pressure control, which has strong theoretical support, is often adapted in RL systems with occupancy-based penalties to avoid gridlock [26, 35, 33]. Switching penalties are also used to prevent frequent light changes, promoting smoother and safer operations [11].

While SUMO’s teleportation feature is not part of the reward, it signals severe congestion or poor system behavior and is treated as a diagnostic tool rather than a control goal [3, 37].

Chapter 3

Methodology and system design

3.1 Simulation Environment and System Setup

This study employs the Simulation of Urban Mobility (SUMO) [37] as the traffic simulation engine. SUMO is a widely adopted open-source microscopic traffic simulator that enables fine-grained modeling of vehicle dynamics, signal control, and network configurations. The simulation framework is integrated with a deep reinforcement learning controller implemented in Python, using the TraCI (Traffic Control Interface) API to facilitate real-time communication between the simulator and the control logic.

3.1.1 Software Dependencies

The experimental framework is built in Python 3.9. It uses SUMO 1.15 for microscopic traffic simulation, and it connects to SUMO through TraCI and sumolib, which are Python tools that let the program read traffic data and control traffic lights. For the learning model, we use PyTorch 2.0, which is a deep learning library.

All Python packages are installed and managed using pip. To make the experiments easy to repeat, we keep the software versions fixed in a configuration file. SUMO must be installed correctly on the computer and available through the environment variable SUMO_HOME. On Linux, SUMO is installed using the official PPA source, while on Windows, it is installed using the official prebuilt installer. In both cases, the SUMO bin folder is added to the system path so that the simulator can be started and controlled by the Python code.

3.1.2 Integration with SUMO

The reinforcement learning controller interacts with SUMO using the TraCI interface, which enables step-wise control of simulation time and access to internal traffic state variables. At each decision step, the controller receives a structured observation vector

containing traffic statistics such as queue length, waiting time, and downstream occupancy. It then computes a control action (e.g., cycle time and phase split) and applies the corresponding signal plan to the network through TraCI. Clearance constraints (yellow and all-red intervals) are respected to ensure signal safety.

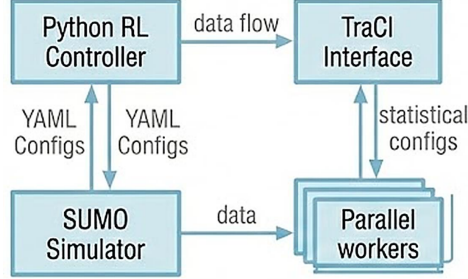


Figure 3.1: System Architecture & Data Flow.

3.1.3 Scenario Configuration

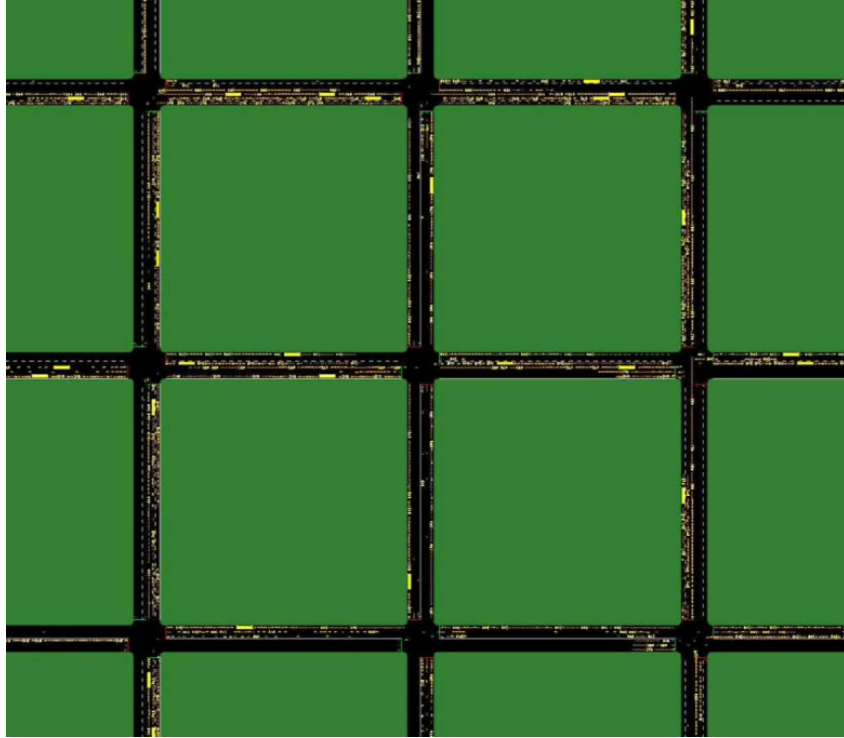


Figure 3.2: Figure 3.2 shows the fixed road network topology used for all experiments.

The traffic scenarios are defined using SUMO’s standard network and route description files. In this study, the road network structure is kept the same for all experiments so that results are comparable under a fixed topology.

SUMO teleportation may occur when the simulator removes vehicles under extreme congestion or gridlock. Since this behavior is specific to simulation, teleport events are

tracked primarily as a diagnostic signal. In the current experimental configuration, the teleport penalty is disabled ($\lambda_{tele} = 0$) to focus the learning strictly on delay minimization and spillback prevention. When downstream sensing is enabled, occupancy information for outgoing links is read through TraCI and used to compute the squared spillback penalty.

3.1.4 System Verification

Correct integration between SUMO and the Python controller is verified through a basic connectivity check. In this check, the system confirms that the TraCI interface can be loaded and that a TraCI connection can be created successfully. This verification step ensures that the simulator can be accessed by the control program and that runtime communication is available. After the interface check, basic system behavior is validated using simple automated checks included in the codebase.

3.2 Training Methodology

This section details the components and strategies used to configure and train the reinforcement learning agent.

3.2.1 MDP specification

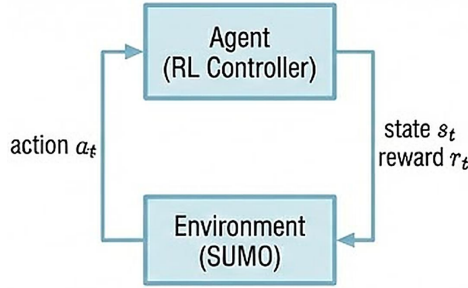


Figure 3.3: MDP Definition.

We define the traffic signal control problem as a Markov Decision Process (MDP) [4]. At each decision step t , the agent observes the current state s_t , selects an action a_t , receives a reward r_t , and transitions to the next state s_{t+1} . The environment follows the traffic dynamics simulated by SUMO, and the agent interacts with it through the TraCI interface [39].

The controller is applied in a multi-intersection setting, using parameter sharing across all controlled traffic lights. Each intersection is treated as an individual agent with

the same observation and action structure. To maintain coordinated behavior across the network, cycle-level constraints are used to ensure consistent signal timing between connected junctions.

State representation

The state is a 12-dimensional vector per traffic light at each decision step,

$$s_t = [q_{N,t}, q_{E,t}, q_{S,t}, q_{W,t}, w_{N,t}, w_{E,t}, w_{S,t}, w_{W,t}, o_{N,t}, o_{E,t}, o_{S,t}, o_{W,t}]^\top \in \mathbb{R}^{12}.$$

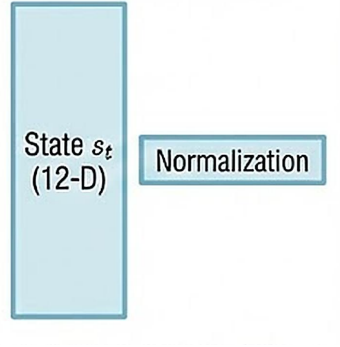


Figure 3.4: State Representation Diagram.

The state vector includes three types of features for each of the four approaches at an intersection. The first four components $q_{.,t}$ represent queue lengths. The next four components $w_{.,t}$ represent total waiting times. The last four components $o_{.,t}$ capture downstream occupancy for outgoing links. Queue and waiting time features help the agent understand local congestion, while downstream occupancy provides information about potential spillback. This design follows common RL-TSC practice using delay, queues, and occupancy as input features [30, 25, 31].

Downstream occupancy is only used when downstream sensing is available and outgoing links are defined. If not, these components are set to zero at each decision step. This keeps the state size fixed while allowing the scenario to switch downstream awareness on or off. Queue counting is based on a per-cycle rule: a vehicle is counted once if it joins a queue at any point during the current decision cycle. A stopped vehicle is identified using a low-speed threshold, which is standard in microscopic traffic simulations to detect queuing behavior.

Normalization

Because queue, waiting time, and occupancy operate on different numeric scales and can vary across curriculum stages, all state features are normalized using fixed mean and standard deviation statistics, with clipping to bounded ranges. This stabilizes optimization

and reduces sensitivity to traffic demand scaling, which is important in deep RL training [4, 15]. The normalization configuration is kept fixed during training and evaluation so that learned policies remain comparable across scenarios.

3.2.2 Action space and constraints

Discrete action design

The action space is discrete with $|\mathcal{A}| = 15$. Each action chooses a cycle length from a set of three options and a split pattern from a set of five options. Let the cycle choices be

$$C \in \{60, 90, 120\} \text{ seconds,}$$

and let each split pattern define ratios (ρ_{NS}, ρ_{EW}) with $\rho_{NS} + \rho_{EW} = 1$. The split options are

$$(\rho_{NS}, \rho_{EW}) \in \{(0.30, 0.70), (0.40, 0.60), (0.50, 0.50), (0.60, 0.40), (0.70, 0.30)\}.$$

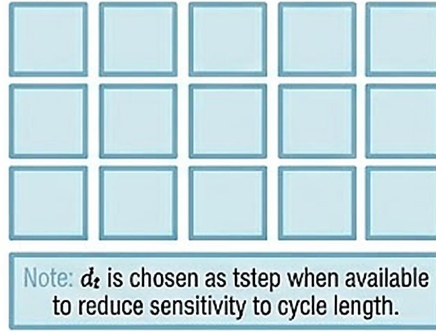


Figure 3.5: Action Space Grid.

An action selects $(C, \rho_{NS}, \rho_{EW})$ and converts the ratios into green durations for the two movement groups.

Green computation with minimum-green safety

Let $g_{\min}$ be the minimum green time in seconds. The raw green allocation is computed as

$$g_{NS}^{raw} = \text{round}(\rho_{NS} \cdot C),$$

then clipped to enforce minimum green feasibility,

$$g_{NS} = \max(g_{\min}, \min(g_{NS}^{raw}, C - g_{\min})), \quad g_{EW} = C - g_{NS}.$$

This ensures both movement groups receive at least $g_{\min}$ seconds of green. In addition to minimum green time, a ratio bound $\rho_{\min}$ is used as a safety bound on extreme splits. The implemented constraint policy enforces feasibility by rejecting or clipping allocations that violate minimum green or minimum ratio bounds. In our implementation, the minimum green time is set to $g_{\min} = 10\text{s}$ and the minimum split ratio bound is $\rho_{\min} = 0.1$ to avoid extreme allocations.

Clearance intervals and decision step duration

Signals include clearance intervals with yellow duration y and all-red duration r . A full decision step includes a full cycle plus clearance transitions for both directions, so the effective decision duration is

$$t_{\text{step}} = C + 2y + 2r.$$

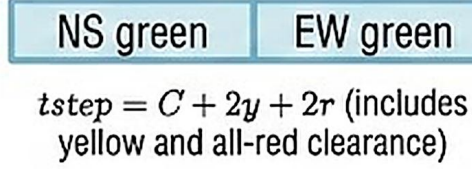


Figure 3.6: Timing Model.

This definition makes reward scaling and time-aware discounting consistent across cycle choices. It also supports stable comparison of policies that choose different cycle settings.

3.2.3 Reward function

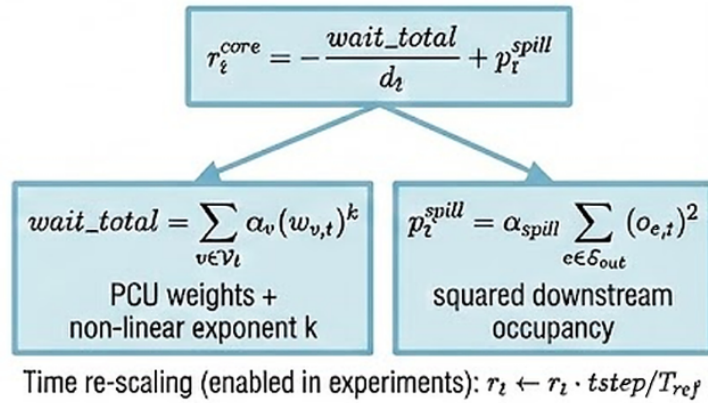


Figure 3.7: Reward Function Breakdown.

Core objective

The primary objective is to reduce delay and congestion. As in many RL-TSC studies, the central term is a penalty proportional to total waiting time or delay [30, 25, 7]. Let *wait_total* denote the aggregated waiting cost over the decision step. The reward is constructed as a negative normalized sum of delay and penalty terms,

$$r_t^{core} = -\frac{wait_total}{d_t} + p_t^{spill},$$

where d_t is a positive normalization denominator. In implementation, d_t is chosen as t_{step} when available; otherwise it falls back to the decision cycle duration, and it is lower-bounded by 1 to avoid numerical instability. This normalization reduces sensitivity to cycle length and keeps reward magnitudes comparable when actions change C .

PCU-weighted and non-linear waiting cost

To better match mixed traffic, waiting contributions can be weighted using passenger car unit (PCU) style weights. Let α_v be the weight for vehicle v , and let k be a non-linearity exponent. The aggregated waiting cost can be expressed as

$$wait_total = \sum_{v \in \mathcal{V}_t} \alpha_v (w_{v,t})^k,$$

where $\mathcal{V}_t$ is the set of vehicles considered in the decision step and $w_{v,t}$ is the waiting measure accumulated over that decision step. The exponent k increases the penalty for large waits and reduces tolerance for starvation. This design is consistent with fairness-motivated reward shaping that discourages extreme delays [11, 31].

Spillback penalty from downstream occupancy

Spillback happens when the roads after an intersection become too full. When this happens, vehicles cannot leave the intersection normally, and traffic starts to block the lanes before the intersection. To reduce this risk, the reward function adds a penalty using the squared downstream occupancy:

$$p_t^{spill} = \alpha_{spill} \sum_{e \in \mathcal{E}_{out}} (o_{e,t})^2,$$

where $o_{e,t}$ is the occupancy of outgoing edge e at time t , and $\alpha_{spill} \geq 0$ is a weight.

This penalty encourages the agent to avoid actions that push too many vehicles into already crowded downstream links. Compared with threshold-based penalties that only activate after occupancy passes a limit, the squared form increases smoothly as occupancy

grows. This gives the agent earlier feedback and encourages it to balance traffic flow before severe congestion or gridlock happens. The idea is consistent with pressure-based traffic control and max-pressure methods [26, 35].

Optional time re-scaling

When time re-scaling is enabled, the computed reward is re-scaled to account for differences between t_{step} and a reference decision duration T_{ref} ,

$$r_t \leftarrow r_t \cdot \frac{t_{\text{step}}}{T_{\text{ref}}}.$$

This is an implementation choice to keep reward magnitudes comparable across decision durations and to reduce learning sensitivity when the policy selects different cycle lengths. In the reported experiments, time re-scaling is enabled to reduce sensitivity to different decision durations.

Time-Aware Discounting

In this setting, actions have different real-time durations, so discounting is applied in a time-aware way. Let γ_0 be the base discount factor for a standard time unit, and let t_{ref} be the reference duration (e.g., 1 second or 5 seconds). For a transition that lasts t_{step} seconds, the actual discount factor applied is:

$$\gamma_t = \gamma_0^{t_{\text{step}}/t_{\text{ref}}}.$$

This follows a semi-Markov interpretation, where discounting depends on the **elapsed real time**, not just on the number of decision steps [4]. The computed discount γ_t is stored together with each transition and used later during learning to compute the target values correctly.

3.2.4 Learning algorithm and network architecture

Dueling Double DQN

The agent uses a value-based deep RL method based on DQN [13] with double Q-learning [14] and a dueling network architecture [40]. The dueling architecture decomposes the action-value function into a state-value term and an advantage term,

$$Q(s, a) = V(s) + \left(A(s, a) - \frac{1}{|\mathcal{A}|} \sum_{a'} A(s, a') \right).$$

This helps learning by separating the value of a state from the relative preference among actions, which is useful when many actions produce similar outcomes in a given traffic regime [40].

Target computation

For a transition (s_t, a_t, r_t, s_{t+1}) with effective discount γ_t , the Double DQN target is

$$y_t = r_t + \gamma_t Q_{\theta^-} \left(s_{t+1}, \arg \max_{a'} Q_{\theta}(s_{t+1}, a') \right),$$

where θ are the online network parameters and θ^- are target network parameters. The loss is the squared temporal difference error,

$$\mathcal{L}(\theta) = \mathbb{E} \left[(Q_{\theta}(s_t, a_t) - y_t)^2 \right].$$

Experience replay and a periodically updated target network stabilize training [13].

Training Stability and Exploration

To ensure stable training, gradient clipping is applied to prevent large parameter updates that may cause divergence. Exploration follows an ϵ -greedy schedule, starting with a high exploration rate that gradually decays to a lower final value. These choices follow standard practices in deep reinforcement learning [13, 15].

Experimental Settings and Hyperparameters

To ensure reproducibility, we report the exact hyperparameters used in our Dueling Double DQN implementation. The network uses two fully connected hidden layers with 256 units each. Optimization is performed using Adam with learning rate 1×10^{-4} . Training uses a replay buffer and a target network update every fixed number of learner steps, following standard DQN stabilization practice [13, 15]. Time-aware discounting is enabled to account for variable real-time duration per decision step, consistent with the semi-MDP treatment described earlier [4].

3.2.5 Curriculum Strategy

Traffic demand is generated procedurally to support a duration-based curriculum training strategy. Instead of varying the traffic density, the traffic load is fixed at a high volume while the episode duration increases across stages. This exposes the controller to longer horizons of accumulated congestion, helping it learn robust policies against gridlock over extended periods.

Table 3.1: Hyperparameters used in the experiment.

Parameter	Value
Hidden layers (units)	[256, 256]
Optimizer	Adam
Learning rate	1×10^{-4}
Batch size	256
Replay buffer size	200,000 transitions
Base discount factor (γ_0)	0.99
Time-aware discounting	enabled
Reference duration (t_{ref})	60.0 seconds
Target update frequency	every 5,000 learner steps
Gradient clipping	$\ g\ \leq 10.0$
Exploration (ϵ)	linear decay from 1.0 to 0.02 over 50,000 steps
Teleport penalty weight (λ_{tele})	0
Spillback weight (α_{spill})	1.0

In Phase 1 (Warmup), the episode duration is set to 1,200 seconds for 200 episodes. In Phase 2 (Learn), the duration is increased to 1,500 seconds for 400 episodes. In Phase 3 (Master), the duration is increased to 1,800 seconds for 600 episodes. In total, the curriculum contains 1,200 training episodes.

3.2.6 Training Procedure Workflow

Training is conducted in an episodic fashion. For each episode, SUMO is initialized with a scenario instance. The agent selects actions at each decision step, and transitions are stored in a replay buffer. Learning is performed using Double DQN with time-aware discounting, where mini-batches are sampled from the replay buffer for parameter updates.

The system can run in a single process or in parallel. Parallel execution is used mainly to collect experience faster. Different parallel workers can also use different ϵ -greedy exploration rates, which helps the agent explore more while keeping the traffic demand consistent with the curriculum design.

3.3 Evaluation Methodology

This section outlines how the trained model is tested and compared against traditional methods.

3.3.1 Baselines

Two baseline controllers are used to evaluate the RL controller. The first baseline is a fixed-time controller. It follows pre-defined signal settings, including a fixed cycle length and fixed phase splits, and it represents a standard traditional method for traffic signal control. The second baseline is a max-pressure controller. This method chooses signal phases based on the pressure difference between incoming and outgoing links, so it tends to serve movements that help reduce network imbalance. Max-pressure control is widely known for strong throughput performance under many traffic conditions [26]. Using both baselines makes the comparison clearer, because it helps separate gains from learning-based adaptivity from gains that come from using a general congestion-aware control rule.

3.3.2 Evaluation Procedure Workflow

During evaluation, the trained policy is executed without exploration. A fixed set of test scenarios is run, and performance is reported using key performance indicators (KPIs) such as average waiting time, maximum waiting time, queue length proxies, throughput, and teleportation counts. Multiple runs with fixed random seeds are used to reduce the effect of stochastic variation [4, 15].

3.3.3 Execution Pipeline for Evaluation

All evaluation uses YAML configuration files. These files keep the main experiment settings in one place, such as the simulation setup, learning settings, reward weights, and the traffic scenarios. This makes the experiments easier to repeat and easier to compare between different runs. During evaluation, the model is tested by running it on a fixed set of scenarios using the same configuration style, so the settings stay consistent.

Chapter 4

Results and Discussion

Simulation results show that the RL-based controller significantly reduces average vehicle waiting time compared to the fixed-time baseline, which is consistent with the delay-minimization goal defined in the reward function. The inclusion of the spillback penalty improves network robustness by discouraging actions that lead to downstream saturation and blocking. This adds a network-awareness component that aligns with principles from pressure-based control [26, 35]. The anti-flicker penalty enhances signal stability by reducing frequent changes in cycle selection, reflecting the known advantages of switching penalties in RL-based traffic control [11].

The results also highlight the value of simulator-specific diagnostics. Although teleportation is not a real-world metric, tracking teleport events helps detect severe congestion and deadlock conditions within the simulation. Penalizing teleportation improves policy robustness and distinguishes real control improvements from artifacts of extreme simulator behavior [37].

Overall, the case study demonstrates that achieving strong performance in RL-based traffic signal control requires more than just applying a learning algorithm. It depends on careful MDP design, constraint handling, normalization, curriculum design, and real-time integration—reflecting the broader conclusions of survey research in this field [31].

Bibliography

Bibliography

- [1] Texas Transportation Institute, “2019 Urban Mobility Report,” 2019.
- [2] P. P. Chakraborty and A. B. Kulkarni, “A Review of Urban Traffic Congestion Models,” *International Journal of Advanced Research in Computer Engineering & Technology*, vol. 4, no. 4, 2015.
- [3] D. Krajzewicz, J. Erdmann, M. Behrisch, and L. Bieker, “Recent Development and Applications of SUMO – Simulation of Urban Mobility,” *International Journal On Advances in Systems and Measurements*, vol. 5, no. 3&4, pp. 128–138, 2012.
- [4] R. Sutton and A. Barto, *Reinforcement Learning: An Introduction*. MIT Press, 2018.
- [5] C. Chen, H. Wei, N. Xu, G. Zheng, M. Yang, Y. Xiong, K. Xu, and Z. Li, “Toward A Thousand Lights: Decentralized Deep Reinforcement Learning for Large-Scale Traffic Signal Control,” in *Proceedings of the AAAI Conference on Artificial Intelligence*, 2020.
- [6] W. Wei, Y. Ma, H. Zhang, and Q. Chen, “Traffic Signal Control for Urban Road Networks Using Adaptive Dynamic Programming,” *IEEE Transactions on Intelligent Transportation Systems*, vol. 15, no. 5, pp. 2344–2355, 2014.
- [7] W. Genders and S. Razavi, “Using a Deep Reinforcement Learning Agent for Traffic Signal Control,” *Transportation Research Part C: Emerging Technologies*, vol. 68, pp. 1–20, 2016.
- [8] K. Abdulhai, R. Pringle, and G. Karakoulas, “Reinforcement Learning for True Adaptive Traffic Signal Control,” *Journal of Transportation Engineering*, vol. 129, no. 3, pp. 278–285, 2003.
- [9] E. Moreno, C. Baladron, J. B. Rosas, A. Aguiar, and J. C. Lopez, “A Reinforcement Learning Based Approach for Traffic Light Control in Smart Cities,” *Sensors*, vol. 21, no. 2, 2021.

- [10] I. Arel, C. Liu, T. Urbanik, and A. Kohls, “Reinforcement Learning-Based Multi-Agent System for Network Traffic Signal Control,” *IET Intelligent Transport Systems*, vol. 4, no. 2, pp. 128–135, 2010.
- [11] X. Liang, X. Du, G. Wang, and Z. Han, “Deep Reinforcement Learning for Traffic Light Cycle Control,” *IEEE Transactions on Vehicular Technology*, vol. 68, no. 2, pp. 1243–1253, 2019.
- [12] T. Chu, J. Wang, L. Codecà, and Z. Li, “Multi-Agent Deep Reinforcement Learning for Large-Scale Traffic Signal Control,” *IEEE Transactions on Intelligent Transportation Systems*, vol. 21, no. 3, pp. 1086–1095, 2020.
- [13] V. Mnih et al., “Human-level Control Through Deep Reinforcement Learning,” *Nature*, vol. 518, pp. 529–533, 2015.
- [14] H. van Hasselt, A. Guez, and D. Silver, “Deep Reinforcement Learning with Double Q-learning,” in *Proceedings of the AAAI Conference on Artificial Intelligence*, 2016.
- [15] V. François-Lavet, P. Henderson, R. Islam, M. G. Bellemare, and J. Pineau, “An Introduction to Deep Reinforcement Learning,” *Foundations and Trends in Machine Learning*, vol. 11, no. 3–4, pp. 219–354, 2018.
- [16] J. G. Wardrop, “Some Theoretical Aspects of Road Traffic Research,” *Proceedings of the Institution of Civil Engineers*, vol. 1, no. 3, pp. 325–362, 1952.
- [17] M. Treiber and A. Kesting, *Traffic Flow Dynamics: Data, Models and Simulation*. Springer, 2013.
- [18] P. Mirchandani and L. Head, “A Real-time Traffic Signal Control System: Architecture, Algorithms, and Analysis,” *Transportation Research Part C*, vol. 9, no. 6, pp. 415–432, 2001.
- [19] P. Hunt, D. Robertson, R. Bretherton, and M. Royle, “The SCOOT On-line Traffic Signal Optimisation Technique,” *Traffic Engineering & Control*, vol. 23, no. 4, pp. 190–192, 1982.
- [20] D. Gartner, “OPAC: A Demand-Responsive Strategy for Traffic Signal Control,” *Transportation Research Record*, no. 906, pp. 75–81, 1983.
- [21] A. Stevanovic, *Adaptive Traffic Control Systems: Domestic and Foreign State of Practice*. NCHRP Synthesis 403, Transportation Research Board, 2010.
- [22] S. El-Tantawy, B. Abdulhai, and H. Abdelgawad, “Multiagent Reinforcement Learning for Integrated Network of Adaptive Traffic Signal Controllers,” *Transportation Research Part C*, vol. 28, pp. 130–151, 2013.

- [23] R. A. Wiering, “Multi-agent Reinforcement Learning for Traffic Light Control,” in Proceedings of the 17th International Conference on Machine Learning, 2000.
- [24] E. Watkins and P. Dayan, “Q-learning,” *Machine Learning*, vol. 8, pp. 279–292, 1992.
- [25] E. van der Pol and F. A. Oliehoek, “Coordinated Deep Reinforcement Learners for Traffic Light Control,” *arXiv:1609.01540*, 2016.
- [26] P. Varaiya, “The Max-Pressure Controller for Arbitrary Networks of Signalized Intersections,” in *Advances in Dynamic Network Modeling in Complex Transportation Systems*, Springer, 2013.
- [27] S. Zheng, J. Li, and L. Zhu, “Cooperative Multi-Agent Reinforcement Learning for Traffic Signal Control,” *IEEE Transactions on Intelligent Transportation Systems*, 2019.
- [28] M. Behrisch, L. Bieker, J. Erdmann, and D. Krajzewicz, “SUMO – Simulation of Urban Mobility: An Overview,” in *SIMUL 2011*, 2011.
- [29] A. Brockman et al., “OpenAI Gym,” *arXiv:1606.01540*, 2016.
- [30] H. Wei, G. Zheng, H. Yao, and Z. Li, “IntelliLight: A Reinforcement Learning Approach for Intelligent Traffic Light Control,” in *Proceedings of the 24th ACM SIGKDD International Conference on Knowledge Discovery & Data Mining*, 2018.
- [31] H. Wei, G. Zheng, V. Gayah, and Z. Li, “A Survey on Traffic Signal Control Methods,” *arXiv preprint arXiv:1904.08117*, 2019.
- [32] K. Abdulhai and K. K. Chan, “Parametric Study on Reinforcement Learning for Traffic Signal Control,” *Journal of Transportation Engineering*, 2001.
- [33] G. Zheng, X. Zang, N. Xu, H. Wei, Z. Yu, V. Gayah, K. Xu, and Z. Li, “Magno: Multi-Agent Reinforcement Learning for Traffic Signal Control,” in *Proceedings of the AAAI Conference on Artificial Intelligence*, 2019.
- [34] L. Li, Y. Lv, and F. Wang, “Traffic Signal Timing via Deep Reinforcement Learning,” *IEEE/CAA Journal of Automatica Sinica*, vol. 3, no. 3, pp. 247–254, 2016.
- [35] Y. Zhang, M. H. Abdoos, and A. Afshar, “Multi-Intersection Traffic Signal Control with Deep Reinforcement Learning,” in *AAAI*, 2020.
- [36] A. K. Yadav, A. Sharma, and R. Singh, “A Review of Traffic Congestion and Control Techniques,” *International Journal of Engineering Research & Technology*, 2014.

- [37] K. C. J. E. H. S. O. J., “Simulation/Teleports,” SUMO Documentation, 2024.
- [38] A. Wongpiromsarn, U. Topcu, and R. M. Murray, “Receding Horizon Control for Multiagent Systems with Temporal Logic Specifications,” Proceedings of the IEEE, 2012.
- [39] M. R. K. B., “TraCI Traffic Lights API,” SUMO Documentation, 2024.
- [40] Z. Wang, T. Schaul, M. Hessel, H. van Hasselt, M. Lanctot, and N. de Freitas, “Dueling Network Architectures for Deep Reinforcement Learning,” in International Conference on Machine Learning (ICML), 2016.